Parallel LLM Inference

Project objectives, requirements and acceptance criteria | Distributed Systems Major Project (Topic 38)

1. Project Overview

This project builds a pipeline-parallel inference system for a small pretrained language model. The model's transformer layers are split across two worker processes that communicate over RPC. A client-facing coordinator accepts generation requests, routes them through the pipeline, and returns generated tokens. The system must preserve per-request ordering and KV-cache state, keep both pipeline stages busy through bounded queues and micro-batching, and recover from a worker crash without returning stale or incorrect output.

No model training is involved. Pretrained weights are loaded as-is; the engineering focus is on partitioning, communication, scheduling, backpressure and fault tolerance. Correctness is defined by token-for-token agreement with single-process inference under greedy decoding.

2. Scope and Assumptions

Item	Decision
Model	GPT-2 small (124M parameters, 12 transformer blocks), loaded from Hugging Face. Optional upgrade: TinyLlama-1.1B or Qwen2.5-0.5B.
Language and libraries	Python 3.10+, PyTorch, Hugging Face *transformers*, *grpcio* / Protocol Buffers, *asyncio*.
Hardware	Ordinary CPUs are sufficient. GPU is optional and must not be required.
Topology	1 coordinator + 2 pipeline workers, each a separate OS process with its own port. Can run on one machine or across two machines.
Decoding	Greedy decoding for all correctness tests (deterministic). Sampling may be added only as an optional feature.
Failure model	Crash-stop failures of a worker process and dropped / delayed RPCs. Byzantine behaviour is out of scope.
Out of scope	Training or fine-tuning, tensor parallelism inside a layer, multi-node GPU clusters, production authentication.

3. Target Architecture

Component	Responsibility
Client	Sends prompts with a unique request ID and max_new_tokens; receives a token stream.
Coordinator	Admits requests into a bounded queue, assigns request IDs, drives the decode loop, forms micro-batches, tracks per-request step numbers, detects worker failure and triggers recovery.
Worker 1 (Stage 0)	Token + position embeddings and transformer blocks 0-5. Holds the KV cache for its layers. Sends hidden-state activations to Worker 2.
Worker 2 (Stage 1)	Transformer blocks 6-11, final layer norm and LM head. Holds the KV cache for its layers. Returns next-token logits / token IDs.

Component	Responsibility
Health monitor	Heartbeats between coordinator and workers; declares a worker dead after a configurable timeout.

Data flow per decode step: **Coordinator -> Worker 1 -> Worker 2 -> Coordinator**. Every message carries (request_id, step, worker_epoch) so that late or duplicated messages can be identified and discarded.

4. Detailed Objectives

Each objective below lists what must be built, the conditions under which it is considered done, and the artefact that demonstrates it. Objectives are ordered so that each one builds on the last.

O1. Single-process baseline and reference outputs

Goal. Establish a ground truth that every later version of the system is checked against.

Requirements

- Load pretrained GPT-2 and run greedy generation for a fixed prompt set (at least 20 prompts of varying length).
- Save the generated token IDs, per-token latency and total runtime to a reference file.
- Pin library versions and set torch.manual_seed and deterministic flags.

Acceptance criteria

- Re-running the baseline produces byte-identical token sequences.
- Reference file is committed to the repository and used by the test suite.

Deliverable. baseline.py and reference_outputs.json.

O2. Layer partitioning across two workers

Goal. Split the model into two pipeline stages that together compute exactly what the full model computes.

Requirements

- Partition point is configurable (default: blocks 0-5 on stage 0, 6-11 on stage 1).
- Each worker loads only the weights it needs and reports its memory footprint.
- Stages expose a forward(hidden_states, request_id, step) interface.

Acceptance criteria

- Running stage 0 then stage 1 in the same process reproduces baseline logits within a tolerance of 1e-5 and identical greedy tokens.
- Memory per worker is approximately half of the full model's weight memory.

Deliverable. partition.py with a unit test comparing split versus full forward passes.

O3. RPC-based activation transfer

Goal. Run the two stages as separate processes and move activations between them over the network.

Requirements

- Define a Protocol Buffers schema for activation tensors (shape, dtype, raw bytes) plus request_id, step and epoch metadata.
- Implement gRPC services on both workers; the coordinator calls stage 0, which forwards to stage 1.

- Measure and log bytes transferred per decode step and per request.
- Handle RPC deadlines: every call has a timeout and a clear error path.

Acceptance criteria

- Distributed generation matches reference_outputs.json token for token for every test prompt.
- Serialisation round-trip of a tensor is lossless (verified by unit test).

Deliverable. Two runnable worker services and a coordinator that generate text end to end.

O4. Per-request KV-cache management

Goal. Avoid recomputing the full prompt on every decode step while keeping requests isolated.

Requirements

- Each worker keeps a KV cache per request_id for its own layers only.
- Prefill step processes the whole prompt once; decode steps send only the newest token's activations.
- Caches are freed on request completion, cancellation or timeout; a cap on total cached requests prevents unbounded memory growth.
- Steps for a request must be applied strictly in order; out-of-order steps are rejected.

Acceptance criteria

- Output with the cache enabled is identical to output without it.
- Per-token decode latency is measurably lower with the cache (report the speed-up).
- Interleaving two requests never mixes their cache entries (dedicated test).

Deliverable. KV-cache module and a comparison table: cache on versus cache off.

O5. Concurrency, bounded queues and backpressure

Goal. Serve many simultaneous requests safely without unbounded memory growth or lost ordering.

Requirements

- Bounded admission queue at the coordinator and bounded input queue at each worker.
- When queues are full, new requests are rejected (or blocked) with an explicit 'overloaded' status rather than silently dropped.
- Per-request ordering is preserved: tokens for one request are emitted in step order.
- Request-level timeout and client cancellation are supported.

Acceptance criteria

- Under a load test of at least 50 concurrent requests, every completed request matches its reference output.
- Queue lengths never exceed configured bounds; memory stays flat under sustained overload.
- Overloaded requests receive an explicit rejection, never a hang.

Deliverable. Load-test script and a plot of queue depth and rejected requests over time.

O6. Micro-batching and pipeline utilisation

Goal. Keep both stages busy so that throughput scales beyond one request at a time.

Requirements

- Group decode steps from several active requests into a micro-batch (configurable max batch size and max wait time).

- Pipeline micro-batches so that stage 0 works on batch k+1 while stage 1 works on batch k.
- Record per-stage busy and idle time.

Acceptance criteria

- Batched output is identical to unbatched output for every request.
- Throughput (tokens/second) with micro-batching is reported against the unbatched pipeline and against the single-process baseline.
- Stage utilisation improves measurably with pipelining enabled.

Deliverable. Benchmark results across batch sizes 1, 2, 4, 8 and 16.

O7. Failure detection and recovery

Goal. Survive a worker crash mid-generation and still return correct output, without stale data leaking through.

Requirements

- Heartbeats detect a dead worker within a configurable timeout.
- On failure, the coordinator restarts the worker with a new, higher epoch number.
- Affected in-flight requests are recovered by replaying prompt plus already-generated tokens to rebuild KV-cache state on the restarted worker.
- Any message carrying an old epoch, or a (request_id, step) already completed, is discarded.
- Retries are idempotent: a step executed twice must not emit a duplicate token.
- Requests that cannot recover within a deadline fail explicitly with an error status.

Acceptance criteria

- Killing either worker mid-generation (via SIGKILL) still yields output identical to the reference.
- No duplicate or missing tokens appear in any recovered response.
- A delayed message from a pre-crash epoch is injected and shown to be rejected.
- Recovery time (detection + restart + replay) is measured and reported.

Deliverable. Fault-injection script and a recovery timeline chart.

O8. Evaluation and benchmarking

Goal. Produce credible, reproducible numbers that describe the system's behaviour.

Requirements

- Report the metrics in Section 5 for the baseline, the pipeline without batching, and the pipeline with micro-batching.
- Vary prompt length, number of concurrent requests and batch size.
- Run each experiment at least 3 times; report mean and p50 / p95 / p99 where relevant.
- One command reproduces all benchmarks and regenerates all plots.

Acceptance criteria

- All numbers in the README can be regenerated from the repository.
- Results include an explanation of where the bottleneck is (compute, serialisation or network).

Deliverable. benchmarks/ directory, raw CSV results and plots.

O9. Documentation and presentation

Goal. Make the project understandable to an interviewer or evaluator in five minutes.

Requirements

- README with architecture diagram, message protocol, guarantees and failure assumptions.
- Explicit statement of guarantees: per-request token ordering, exact match with baseline under greedy decoding, behaviour on overload and on unrecoverable failure.
- Short demo recording: normal generation, overload, and worker kill with recovery.

Acceptance criteria

- A new reader can set up and run the system from the README alone.
- Design decisions and their trade-offs are written down, including what was left out.

Deliverable. README.md, design document and demo GIF or video.

5. Metrics to Report

Metric	Definition	Why it matters
Time to first token	Time from request admission to first generated token	User-perceived responsiveness; dominated by prefill
Inter-token latency	Mean and p95 time between consecutive tokens	Smoothness of streaming output
Throughput	Total generated tokens per second across all requests	Capacity of the system
Bytes transferred	Activation bytes per token and per request	Communication cost of pipeline parallelism
Stage utilisation	Fraction of time each worker is computing	Shows pipeline bubbles and imbalance
Rejection rate	Share of requests rejected under overload	Effectiveness of backpressure
Recovery time	Failure detection + restart + cache replay	Fault-tolerance cost
Correctness	Fraction of responses matching the reference exactly	Must be 100%

6. Required Experiments and Demonstrations

#	Scenario	Expected result
E1	Single request, distributed vs. baseline	Identical tokens; latency overhead quantified
E2	KV cache on vs. off	Identical tokens; lower per-token latency with cache
E3	1 to 50+ concurrent requests	Throughput curve; all outputs correct
E4	Batch size sweep (1, 2, 4, 8, 16)	Throughput vs. latency trade-off curve
E5	Overload beyond queue capacity	Explicit rejections; bounded memory
E6	Kill Worker 1 mid-generation	Recovery; output matches reference
E7	Kill Worker 2 mid-generation	Recovery; output matches reference
E8	Inject a stale-epoch / duplicate message	Message discarded; no duplicate tokens
E9	Artificial network delay between workers	Effect on latency and throughput explained

7. Milestones

Estimated for one developer working 2-4 hours a day with AI assistance; roughly 1.5-2.5 weeks in total.

Phase	Objectives	Estimate	Exit condition
1. Baseline	O1	0.5 day	Reference outputs saved
2. Pipeline split over RPC	O2, O3	1-2 days	End-to-end match with baseline
3. KV cache	O4	1 day	Cache on/off outputs identical
4. Concurrency and batching	O5, O6	2-3 days	Load test passes; throughput plot
5. Failure recovery	O7	2-3 days	Kill tests pass; no duplicates
6. Benchmarks and docs	O8, O9	1-2 days	README, plots and demo complete

8. Stretch Goals (Optional)

- Extend from 2 to N pipeline stages with automatic, memory-balanced layer assignment.
- Continuous batching: admit new requests into a running batch at every decode step.
- Activation compression (fp16 or int8) and its effect on bytes transferred and accuracy.
- Run the two workers on separate machines and compare LAN versus localhost.
- Replace GPT-2 with a modern architecture (TinyLlama or Qwen2.5-0.5B).
- Token streaming to the client over gRPC server streaming or WebSockets.

9. Distributed Systems Concepts Demonstrated

Model (pipeline) parallelism; RPC and data serialisation; pipelining and stage utilisation; bounded queues and backpressure; per-request ordering; soft state (KV cache) and its reconstruction; failure detection with heartbeats; epochs and stale-message rejection; idempotent retries; communication-cost analysis.

10. Example Resume Bullet

Built a pipeline-parallel LLM inference system in Python (PyTorch, gRPC) splitting GPT-2 across worker processes with per-request KV caching, micro-batching and backpressure; achieved [X]x throughput over unbatched serving and recovered from worker crashes in [Y] ms with 100% token-exact output. Fill in the bracketed numbers from your own benchmarks.